

Capítulo 4

Subsistema de Memoria Compartida: Coherencia Cache

4.1 Introduccion y motivacion

Conexion con temas anteriores Temas 1–3

El Tema 1 presento los procesadores multinucleo: varios nucleos comparten el mismo chip, el mismo espacio de direcciones y, en muchos casos, niveles de cache L3. El Tema 2 explico las caches L1/L2 privadas por nucleo y la L3 compartida. El Tema 3 mostro que los procesadores modernos ejecutan instrucciones especulativamente y en paralelo. Ahora surge la pregunta inevitable: si varios nucleos mantienen copias privadas del mismo dato en sus caches, que ocurre cuando uno de ellos lo modifica? Los demas nucleos leerian el valor antiguo? El Tema 4 responde esta pregunta con los **protocolos de coherencia de cache**.

En un sistema multiprocesador de memoria compartida, cada procesador dispone de su propia **cache privada**. Esta estructura es fundamental para el rendimiento: sin cache, cada acceso a memoria principal anadiriamos cientos de ciclos de latencia. Sin embargo, la cache privada introduce el problema de la **coherencia**: si dos o mas procesadores guardan copias del mismo bloque de memoria y uno de ellos lo modifica, las demas copias quedan *obsoletas*, esto es, representan un valor que ya no es correcto.

4.1.1 Tipos de datos en las caches de un multiprocesador

En un sistema multiprocesador distinguimos dos categorias segun el patron de acceso a los datos:

- **Datos privados:** usados exclusivamente por un unico procesador (variables locales de un hilo, la pila). No generan problemas de coherencia porque ningun otro procesador los leera ni modificara.
- **Datos compartidos:** usados por varios procesadores (estructuras de datos globales, resultados que los hilos intercambian). Cuando un procesador modifica su copia local de un dato compartido, los demas procesadores que tambien tienen una copia en su cache quedan con informacion incorrecta. **Este es el problema de coherencia.**

4.1.2 Escenario concreto de incoherencia: ejemplo paso a paso

Para que quede absolutamente claro que significa la incoherencia, sigamos un escenario con tres procesadores P1, P2 y P3, cada uno con su cache privada, compartiendo una memoria principal conectada por un bus:

1. **Estado inicial:** la variable D tiene valor 3 en memoria principal. Ninguna cache la contiene aun.
2. **P1 y P2 leen D :** ambos cargan el bloque que contiene D en sus caches respectivas. Cada cache tiene una copia con valor 3. La memoria tambien tiene valor 3. El sistema es coherente.
3. **P1 escribe $D \leftarrow 34$** (usando politica de post-escritura, *write-back*): P1 actualiza solo su copia local a 34. La cache de P2 sigue con valor 3. La memoria principal sigue con valor 3. **El sistema es ahora INCOHERENTE.**
4. **P2 lee D :** P2 obtiene 3 desde su cache, un valor obsoleto. Lee un dato incorrecto.
5. **P3 lee D :** P3 falla en su cache y acude a memoria principal, que tampoco tiene el valor actualizado. P3 tambien lee 3 (incorrecto).

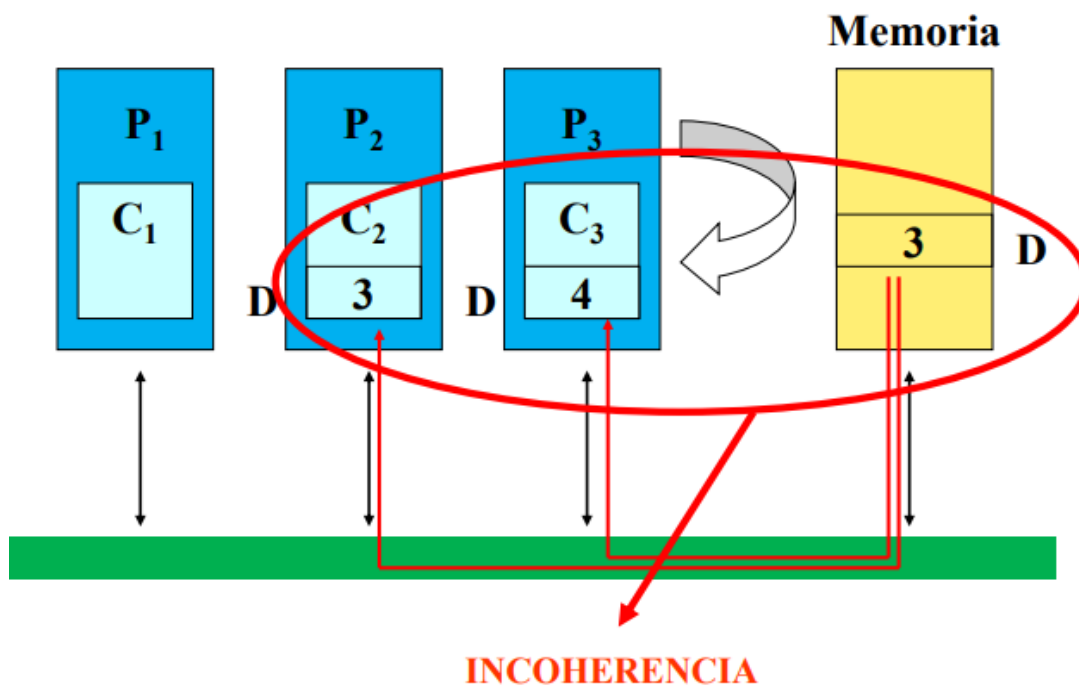


Figura 4.1: Escenario de incoherencia de cache. P1 modifico D en su cache pero no propago el cambio. P2 y la memoria principal tienen copias obsoletas del dato.

Atencion La incoherencia ocurre con ambas politicas de escritura

Con **escritura directa** (write-through): P1 actualiza su cache y la memoria principal al mismo tiempo. La memoria queda correcta, pero la cache de P2 queda obsoleta. Hay incoherencia entre caches aunque no entre cache y memoria.

Con **post-escritura** (write-back): P1 solo actualiza su cache. La memoria y la cache de P2 quedan obsoletas. Hay incoherencia entre caches y tambien entre cache y memoria. La situacion es peor que con escritura directa.

La conclusion es que **ambas politicas generan incoherencia** en sistemas multiprocesador. La solucion requiere protocolos hardware especificos independientemente de la politica de escritura elegida.

4.2 Definiciones formales: coherencia y consistencia

Definicion Coherencia de memoria

Un sistema de memoria es **coherente** si cualquier lectura de una direccion, sea cual sea el procesador que la realice, devuelve el **valor mas reciente** escrito para esa direccion. La coherencia exige tres propiedades:

1. **Preservar el orden del programa:** si un procesador P escribe la posicion X y luego P lee la posicion X (sin escrituras intermedias de otro procesador), el valor devuelto es siempre el valor escrito por P .
2. **Vista coherente de la memoria:** si un procesador Q escribe la posicion X y luego otro procesador P lee X , el valor devuelto es el valor escrito por Q , siempre que no haya escrituras intermedias.
3. **Serializacion de escrituras:** dos escrituras a la misma posicion por dos procesadores cualesquiera son vistas en el *mismo orden* por todos los procesadores del sistema. El bus compartido garantiza esta propiedad de forma natural al serializar todas las transacciones.

Definicion Consistencia de memoria

La **consistencia de memoria** define el orden en que las operaciones a *distintas* posiciones de memoria son ejecutadas unas respecto a otras. Garantiza que las operaciones se realizan en el orden del programa secuencial. Coherencia y consistencia son conceptos distintos pero complementarios: la coherencia garantiza que todos ven el mismo valor para una misma direccion; la consistencia garantiza el orden entre operaciones a distintas direcciones.

4.3 Multiprocesadores UMA / SMP

Definicion Multiprocesador UMA / SMP

Un **multiprocesador UMA** (*Uniform Memory Access*), tambien llamado **SMP** (*Symmetric MultiProcessor*), es un sistema en el que todos los procesadores comparten una unica memoria principal con la misma latencia de acceso para todos ellos. Caracteristicas:

- Numero de nucleos pequeno (tipicamente ≤ 8).

- Todos los procesadores comparten un **bus comun**.
- Cada procesador tiene su cache privada.
- Latencia de acceso a memoria uniforme para todos.

Son los llamados multiprocesadores simetricos. Los procesadores multinucleo modernos en un unico chip son esencialmente SMPs integrados en silicio.

4.4 Metodos de propagacion de escrituras: WU y WI

Los protocolos de coherencia se basan en propagar a todas las caches los efectos de cada escritura. Existen dos enfoques fundamentales:

4.4.1 Escritura con actualizacion (WU, Write-Update)

Cuando un procesador escribe en un dato compartido, el nuevo valor se **distribuye inmediatamente** a todas las caches que tienen copia. Actua como escritura directa para datos compartidos y como post-escritura para datos privados.

Desventajas: multiples escrituras a la misma palabra generan multiples difusiones; en lineas de cache de varias palabras, cada palabra escrita genera una difusion. El trafico de bus puede ser muy alto.

4.4.2 Escritura con invalidacion (WI, Write-Invalidate)

Definicion Escritura con invalidacion (WI)

Cuando un procesador escribe en un dato compartido, envia una **senal de invalidacion** a traves del bus. Todas las caches que tienen copia de ese bloque la marcan como **invalida**. El procesador que escribe pasa a ser el unico con copia valida.

Es la solucion mas comun. Se implementa mediante los **protocolos de snooping**. Ventajas clave:

- Múltiples escrituras al mismo bloque (sin lecturas entre medias) generan solo *una* invalidacion inicial; las siguientes escrituras no generan trafico de bus.
- Solo la primera escritura a cualquier palabra de la linea invalida toda la linea; escrituras posteriores a otras palabras no generan trafico.

Cuadro 4.1: Comparativa WU vs WI

Situacion	WU	WI
Multiples escrituras a la misma palabra	Una difusion por escritura	Solo la primera invalida
Multiples escrituras a distintas palabras de la misma linea	Una difusion por palabra	Solo la primera escritura invalida la linea
Retardo escritura P – lectura Q	Bajo (actualizacion inmediata)	Mayor (Q debe recargar el bloque)
Nivel de actuacion	Palabra	Linea de cache

Cuadro 4.2: Snooping vs. protocolos basados en directorio

	Snooping (Espionaje)	Basado en directorio
Arquitectura tipica	UMA/SMP con bus compartido	CC-NUMA con red escalable
Mecanismo	Cada cache espía el bus para detectar operaciones	Directorio gestiona el estado de cada bloque
Tipo de comunicacion	Difusion (broadcast)	Punto a punto solo entre implicados
Escalabilidad	Baja (el bus es cuello de botella)	Alta
Ejemplo	Intel Core i7 (intra-chip, entre nucleos)	AMD Opteron, servidores NUMA

4.5 Tipos de protocolos de coherencia

4.6 Protocolos de snooping: principio de funcionamiento

Definición Snooping

En un **protocolo de snooping**, cada controlador de cache *espía* el bus para detectar todas las operaciones de los demás procesadores. Cuando un procesador emite una escritura, el resto detectan la señal en el bus y actualizan o invalidan sus copias según el protocolo.

El mecanismo de invalidación mediante snooping funciona en cinco pasos:

1. Un procesador quiere escribir en una dirección. Adquiere el bus (uso exclusivo).
2. Difunde la dirección a invalidar a través del bus.
3. Todos los demás procesadores escuchan el bus (*snooping*).
4. Cada uno comprueba si tiene en su cache un bloque con esa dirección.
5. Si lo tiene, marca la línea como inválida (bit de validez $V = 0$).

La **serialización del bus** garantiza la tercera condición de coherencia de forma natural: como solo un procesador puede usar el bus en cada instante, no pueden producirse dos escrituras simultáneas al mismo bloque.

4.7 Protocolo MSI: tres estados

4.7.1 Los tres estados

El **protocolo MSI** es el protocolo de snooping con invalidación más simple. Usa **post-escritura** como política de actualización de memoria principal. Cada bloque de cache puede estar en uno de tres estados:

Estado M — MODIFICADO

Este procesador es el **único** que tiene una copia válida del bloque. La memoria principal y todas las demás caches tienen copias **obsoletas** o no tienen copia. Las escrituras locales son aciertos sin tráfico de bus.

Obligaciones cuando se espía el bus:

- Otro proc. solicita el bloque (PtLec): depositar en bus (RpBloque) y pasar a **S**. Memoria recoge el bloque.
- Otro proc. solicita acceso exclusivo (PtLecEx): depositar en bus, memoria recoge e invalida, esta cache pasa a **I**.

Estado S — COMPARTIDO (Shared)

El bloque esta presente en **varias caches** y **todas** las copias estan actualizadas, igual que la memoria principal. Las lecturas locales son aciertos sin trafico.

Obligaciones cuando se espía el bus:

- Otro proc. solicita acceso exclusivo (PtLecEx): invalidar la copia local, pasar a **I**.

Estado I — INVALIDO

El bloque **no esta** en la cache o ha sido invalidado. Su contenido no es fiable. Cualquier acceso local genera un fallo y desencadena una peticion en el bus.

4.7.2 Mensajes del bus en MSI

Cuadro 4.3: Mensajes del bus en el protocolo MSI

Mensaje	Generado por	Descripcion
PrLec	Procesador local	Lectura local
PrEsc	Procesador local	Escritura local
PtLec	Controlador cache	Peticion de lectura: fallo de lectura. Pone en bus la direccion del bloque.
PtLecEx	Controlador cache	Peticion de acceso exclusivo (fallo o acierto de escritura en S o I). Invalida copias en otras caches.
PtPEsc	Controlador cache	Post-escritura al reemplazar un bloque en M. Transfiere el bloque a memoria.
RpBloque	Cache con bloque en M	Respuesta con el bloque solicitado cuando la cache tiene la unica copia valida (estado M).

4.7.3 Tabla de transiciones de estado MSI

Cuadro 4.4: Tabla completa de transiciones del protocolo MSI

Estado	Evento	Accion	Sig. estado
M	PrLec	Acierto, sin accion en bus	M
M	PrEsc	Acierto, sin accion en bus	M

(Continuacion tabla MSI)

Estado	Evento	Accion	Sig. estado
M	PtLec en bus	RpBloque; memoria recoge el bloque	S
M	PtLecEx en bus	RpBloque; memoria recoge; invalida copia local	I
M	Reemplazo	PtPEsc (post-escritura a memoria)	I
S	PrLec	Acierto, sin accion en bus	S
S	PrEsc	Generar PtLecEx; invalida otras copias	M
S	PtLec en bus	Sin accion (memoria sirve el bloque)	S
S	PtLecEx en bus	Invalida la copia local	I
I	PrLec	Fallo; generar PtLec; esperar RpBloque o mem	S
I	PrEsc	Fallo; generar PtLecEx; obtener bloque exclusivo	M
I	PtLec / PtLecEx en bus	Sin accion (no hay copia)	I

4.7.4 Descripcion detallada de cada escenario MSI

Fallo de lectura: estado I $\rightarrow$ S

El procesador quiere leer un bloque que no esta en su cache (estado I). El controlador genera **PtLec** en el bus.

- Si el bloque esta en **M** en otra cache: esa cache deposita el bloque en el bus (RpBloque) y pasa a S. La memoria recoge el bloque y queda valida. La cache solicitante pasa a S.
- Si el bloque esta en **S** en otra cache: la memoria sirve el bloque. Todas las copias permanecen en S.
- Si el bloque solo esta en memoria: la memoria sirve el bloque. La cache solicitante pasa a S.

Fallo de escritura: estado I $\rightarrow$ M

El procesador quiere escribir en un bloque que no tiene en cache. Genera **PtLecEx** en el bus, indicando que necesita el bloque con acceso exclusivo:

- Si la memoria tiene el bloque valido: lo deposita en el bus y lo marca como invalido.
- Si otra cache tiene el bloque en M: lo deposita (RpBloque) y pasa a I.

Petición	Estado	Acción	Descripción
Acierto de lectura	$S \rightarrow S$	Acierto	Leer dato de caché local
Acierto de lectura	$M \rightarrow M$	Acierto	Leer dato de caché local
Fallo de lectura	$I \rightarrow S$	Fallo	Difundir fallo de lectura en bus.
Fallo de lectura	$S \rightarrow S$	Reemplazo	Fallo de conflicto de dirección. Difundir fallo de lectura en bus.
Fallo de lectura	$M \rightarrow S$	Reemplazo	Fallo de conflicto de dirección. Escribir bloque y difundir fallo de lectura.
Acierto de escritura	$M \rightarrow M$	Acierto	Escribir dato en caché local.
Acierto de escritura	$S \rightarrow M$	Coherencia	Invalidación en bus.
Fallo de escritura	$I \rightarrow M$	Fallo	Difundir fallo de escritura en bus.
Fallo de escritura	$S \rightarrow M$	Reemplazo	Fallo de conflicto de dirección. Difundir fallo de escritura en bus.
Fallo de escritura	$M \rightarrow M$	Reemplazo	Fallo de conflicto de dirección. Escribir bloque y difundir fallo de escritura.

Figura 4.2: Automata del protocolo MSI. Las transiciones en *italica* son eventos observados en el bus generados por otros procesadores; las demas son acciones del procesador local.

- Si otra cache tiene el bloque en S: lo invalida (pasa a I).

La cache solicitante recibe el bloque y pasa a estado M.

Acierto de escritura en bloque compartido: $S \rightarrow M$

El procesador escribe en un bloque en S (acierto: el bloque esta en cache). Aunque es un acierto, debe obtener acceso exclusivo antes de poder modificarlo. Genera **PtLecEx**, que invalida todas las copias en otras caches. El estado pasa a M. No hay distincion entre acierto y fallo de escritura en MSI: ambos generan PtLecEx cuando el estado no es M.

Acierto de escritura en bloque modificado: $M \rightarrow M$

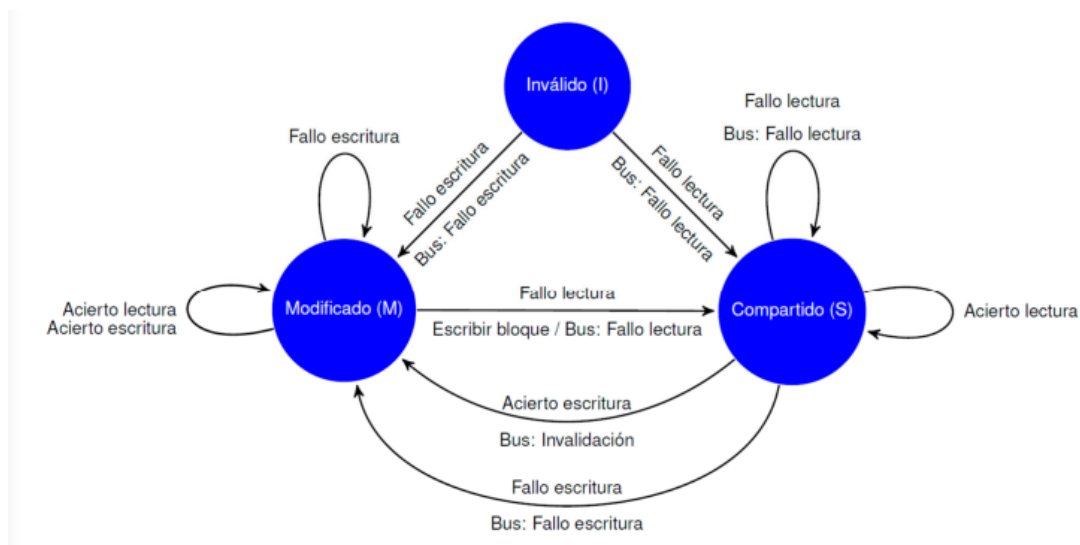
El procesador escribe en un bloque ya en M. Ya tiene acceso exclusivo. No se genera ningun mensaje en el bus. El estado permanece en M.

Acierto de lectura

El bloque esta en M o S (valido). No se genera ninguna accion en el bus. El estado no cambia.

Reemplazo de bloque modificado: $M \rightarrow I$

Al cargar un nuevo bloque, si el bloque a reemplazar esta en M, el controlador genera **PtPEsc**, transfiriendo el bloque a memoria (que pasa de invalido a valido). La linea queda en estado I en la cache.



Estado inicial	Evento	Acción	Estado sig.
Modificado (M)	Procesador lee (PrLec)		Modificado
	Procesador escribe (PrEsc)		Modificado
	Petición de lectura (PtLec)	RpBloque	Compartido
	Petición de acceso exclusivo (PtLecEx)	RpBloque Invalida bloque	Inválido
	Reemplazo	PtPEsc	Inválido
Compartido (S, Shared)	Procesador lee (PrLec)		Compartido
	Procesador escribe (PrEsc)	PtLecEx	Modificado
	Petición de lectura (PtLec)		Compartido
	Petición de acceso exclusivo (PtLecEx)	Invalida bloque	Inválido
Inválido (I)	Procesador lee (PrLec)	PtLec	Compartido
	Procesador escribe (PrEsc)	PtLecEx	Modificado
	Petición de lectura (PtLec)		Inválido
	Petición de acceso exclusivo (PtLecEx)		Inválido

RpBloque es Replicar el bloque (bloque = línea caché)

Figura 4.3: Protocolo MSI: Acciones del procesador

4.8 Protocolo MESI: cuatro estados

4.8.1 Motivacion: ineficiencia del MSI con datos privados

En MSI, cuando un procesador usa un bloque que ningun otro procesador tiene en cache (un dato privado), el flujo es: $I \rightarrow S$ al leer, y luego genera PtLecEx al escribir (para pasar a M), aunque no haya ningun otro nodo que necesite invalidar. Esta PtLecEx es completamente innecesaria y genera trafico de bus inutil.

Concepto clave El estado E elimina la invalidacion innecesaria

El protocolo MESI anade el estado **E (Exclusivo)**: la cache tiene la unica copia del bloque Y la memoria principal tambien tiene copia valida. Al escribir en un bloque en E, la cache puede pasar directamente a M **sin generar ninguna senal en el bus**, porque sabe que no hay otras copias que invalidar. Esta transicion $E \rightarrow M$ es completamente silenciosa.

4.8.2 Los cuatro estados del protocolo MESI

Estado M — MODIFICADO

Unica cache con copia valida. **Memoria obsoleta.** Igual que en MSI. Escrituras locales son aciertos sin trafico. Al espiar PtLec en bus: RpBloque y pasar a S. Al espiar PtLecEx: RpBloque, invalida, pasar a I.

Estado E — EXCLUSIVO

Unica cache con copia valida. **Memoria actualizada.** La diferencia clave con M es que la memoria es valida.

Implicacion crucial:

- Escritura local en E: pasar a M **SIN generar ninguna senal en el bus**. No hay otras copias que invalidar.
- Reemplazo de bloque en E: no requiere post-escritura (la memoria ya es valida).
- Al espiar PtLec en bus: pasar a S (otra cache obtiene copia).
- Al espiar PtLecEx en bus: invalida la copia local, pasar a I.

Estado S — COMPARTIDO

Bloque valido en esta cache, en memoria y en al menos otra cache. Igual que en MSI. Escrituras locales requieren PtLecEx para pasar a M.

Estado I — INVALIDO

Bloque no presente o invalidado. Igual que en MSI. Cualquier acceso local genera fallo y peticion en el bus.

Cuadro 4.5: Resumen de los cuatro estados MESI

Estado	Descripcion	Unica copia en cache?	Memoria valida?
M	Unica copia, memoria obsoleta	Si	No
E	Unica copia, memoria actualizada	Si	Si
S	Multiples copias, todas validas	No	Si
I	No presente o invalidado	—	—

4.8.3 Tabla de transiciones MESI

Cuadro 4.6: Tabla de transiciones del protocolo MESI

Estado	Evento	Accion	Sig. estado
M	PrLec / PrEsc	Acierto, sin accion	M
M	PtLec bus	RpBloque; memoria recoge	S
M	PtLecEx bus	RpBloque; invalida	I
M	Reemplazo	PtPEsc a memoria	I
E	PrLec	Acierto, sin accion	E
E	PrEsc	Pasar a M SIN senal en bus	M
E	PtLec bus	Pasar a S	S
E	PtLecEx bus	Invalida copia local	I
E	Reemplazo	Sin post-escritura	I
S	PrLec	Acierto, sin accion	S
S	PrEsc	Generar PtLecEx	M
S	PtLec bus	Sin accion	S
S	PtLecEx bus	Invalida	I
I	PrLec (sin otras copias)	Fallo; PtLec; unica copia	E
I	PrLec (con otras copias)	Fallo; PtLec; compartida	S
I	PrEsc	Fallo; PtLecEx	M
I	PtLec / PtLecEx bus	Sin accion	I

Estado inicial	Evento	Acción	Estado sig.
Modificado (M)	Procesador lee (PrLec)		Modificado
	Procesador escribe (PrEsc)		Modificado
	Petición de lectura (PtLec)	RpBloque	Compartido
	Petición de acceso exclusivo (PtLecEx)	RpBloque Invalida bloq.	Inválido
	Reemplazo 12	PtPEsc	Inválido
Exclusivo (E)	Procesador lee (PrLec)		Exclusivo
	Procesador escribe (PrEsc)		Modificado

Estado inicial	Evento	Acción	Estado sig.
Compartido (S, Shared)	Procesador lee (PrLec)		Compartido
	Procesador escribe (PrEsc)	PtLecEx	Modificado
	Petición de lectura (PtLec)		Compartido
	Petición de acceso exclusivo (PtLecEx)	Invalida copia local	Inválido
Inválido (I)	Procesador lee (PrLec) Hay copias en otras cachés	PtLec	Compartido
	Procesador lee (PrLec) No hay copias en otras cachés	PtLec	Exclusivo
	Procesador escribe (PrEsc)	PtLecEx	Modificado
	Petición de lectura (PtLec)		Inválido
	Petición de acceso exclusivo (PtLecEx)		Inválido

Figura 4.4: Automata del protocolo MESI. La transición $E \rightarrow M$ es silenciosa (sin tráfico de bus), que es la optimización clave frente a MSI.

Cuadro 4.7: Extensiones del protocolo MESI en procesadores comerciales

Protocolo	Estado extra	Significado	Procesador
MESI	E (Exclusivo)	Única copia, memoria válida	Base
MESIF	F (Forward)	Variante de S: este nodo responde a peticiones	Intel Core i7
MOESI	O (Owned)	Desactualizado en memoria, varias caches tienen copia; evita escrituras a memoria	AMD Opteron

4.8.4 Extensiones comerciales de MESI

4.9 Fallos de coherencia: comparticion verdadera y falsa

Nota de examen Este concepto es central en los ejercicios de traza

La distincion entre fallo de comparticion verdadera y falsa comparticion es el nucleo de los ejercicios de traza del examen. Dominarla es imprescindible.

Los protocolos de invalidacion generan dos tipos de fallos de cache adicionales a los clasicos (obligatorio, capacidad, conflicto):

Fallo de comparticion verdadera (True Sharing Miss)

Se produce cuando:

1. El procesador P1 escribe en un bloque compartido e invalida copias en otras caches.
2. El procesador P2 necesita leer **la misma variable** que P1 escribio.
3. P2 tiene un fallo porque su copia fue invalidada, y debe obtener el nuevo valor de P1.

Este fallo es **inevitable**: representa una comunicacion real de datos entre procesadores. P2 genuinamente necesita el valor que P1 calculo.

Fallo de falsa comparticion (False Sharing Miss)

Se produce cuando:

1. Dos variables sin relacion logica entre si, por ejemplo w_1 usada solo por P1 y w_2 usada solo por P2, residen en el **mismo bloque de cache** (misma linea).
2. P1 escribe w_1 . La linea completa se invalida en la cache de P2 (aunque P2 no necesita w_1).
3. P2 accede a w_2 . Tiene un fallo porque la linea fue invalidada. Debe recargar toda la linea aunque w_2 no cambio.

Este fallo es **evitable** con una alineacion de datos adecuada. No hay comunicacion real de datos, solo una colision accidental en la granularidad de la cache: el bloque se invalida por completo aunque solo una parte fue modificada. La solucion de software es colocar variables frecuentemente escritas por distintos hilos en bloques de cache separados (*padding* o alineacion a tamano de linea de cache).

Atencion Distincion clave para el examen

Un fallo es de **falsa comparticion** cuando el procesador falla en cache a causa de una invalidacion por la escritura de una variable *distinta* que estaba en el mismo bloque. Un fallo es de **comparticion verdadera** cuando el procesador falla porque necesita leer precisamente la variable que otro procesador modifico.

4.10 Limitaciones del snooping y protocolos de directorio

4.10.1 Cuello de botella del bus

El bus compartido y el ancho de banda que requiere el snooping son el **cuello de botella** que limita la escalabilidad de los SMPs. Cada escritura genera al menos un mensaje de invalidacion difundido a todos los procesadores. Con muchos procesadores, el bus queda saturado.

4.10.2 Protocolos basados en directorio

Definicion Directorio de coherencia

Un **directorio de coherencia** mantiene para cada bloque de memoria fisica el estado de comparticion: que caches tienen copia y si alguna lo ha modificado. En lugar de difundir las invalidaciones a todos, el directorio envia mensajes **punto a punto** solo a los procesadores que tienen copia. Esto elimina la necesidad de un bus compartido y permite escalar a muchos mas procesadores.

Los protocolos de directorio se usan en:

- **Sistemas CC-NUMA** (*Cache-Coherent Non-Uniform Memory Access*): multiprocesadores con memoria fisicamente distribuida. Cada nodo tiene su memoria local pero puede acceder a toda la memoria a traves de una red de interconexion. No existe un bus compartido unico.
- **SMPs con red escalable**: cuando el numero de procesadores supera lo que un bus unico puede soportar.
- **Chips multicore con cache L3 compartida**: el directorio centralizado gestiona la coherencia entre las caches privadas L1/L2 de los distintos nucleos. Ejemplo: **Intel Core i7**.

4.10.3 Estados del directorio

Para cada bloque de memoria, el directorio mantiene uno de tres estados:

4.10.4 Vector de bits de presencia

La informacion de que caches tienen copia se almacena como un **vector de N bits** (uno por nodo): bit = 1 indica que esa cache tiene una copia valida. Cuando hay que

Cuadro 4.8: Estados de un bloque en el directorio de memoria

Estado	Significado	Informacion en el directorio
Local	El bloque esta actualizado en memoria; no hay copias en caches	Sin bits de presencia activos
Compartido	Multiples copias validas en caches; memoria actualizada	Vector de bits de presencia: que caches tienen copia
Exclusivo	Una copia en cache en M o E; memoria puede estar obsoleta	Identificador del unico nodo con la copia

invalidar, el directorio solo envia mensajes a los nodos con bit = 1, en lugar de difundir a todos.

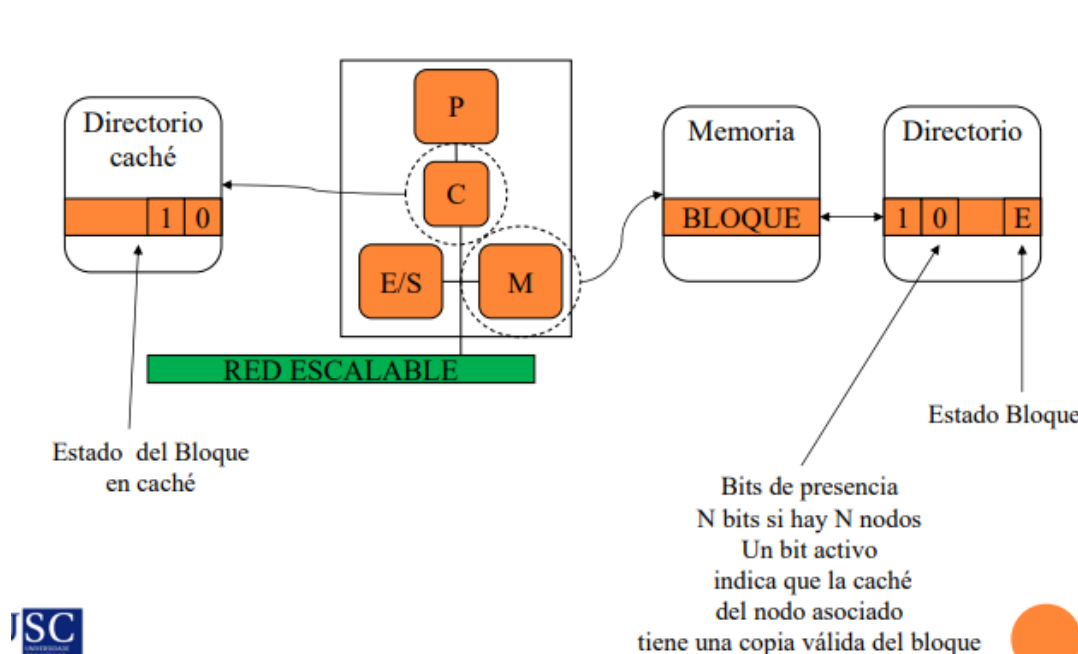


Figura 4.5: Directorio con vector de bits completo. Solo se mandan invalidaciones a los procesadores con bit = 1, eliminando la difusión del snooping.

4.10.5 Clasificación de los directorios

- **Directorio centralizado:** una única tabla con entrada por bloque de memoria. Cuello de botella (todas las peticiones pasan por un punto).
- **Directorio distribuido entre módulos de MP:** cada módulo de memoria tiene su subdirectorio. Los subdirectorios atienden en paralelo. Variantes:

- *Vector de bits completo*: N bits por bloque, uno por nodo. Preciso pero ocupa mucho para N grande.
- *Vector por grupos*: un bit por grupo de nodos. Menos memoria pero menos precision (puede generar invalidaciones extras).
- *Directorio limitado*: solo almacena K nodos con copia. Si hay mas de K, invalida algunos.
- **Directorio encadenado (*linked list*)**: la informacion se distribuye entre las propias caches formando una lista enlazada. El directorio en memoria solo almacena el puntero al primer nodo.

4.11 Comparativa final de protocolos

Cuadro 4.9: Resumen comparativo completo

Criterio	Snooping	Directorio
Comunicacion	Difusion (broadcast) a todos	Punto a punto, solo implicados
Escalabilidad	Baja (≤ 8 nucleos tipico)	Alta (cientos de nodos)
Latencia base	Baja	Mayor (consultar directorio)
Overhead de memoria	Bajo	Alto (vector de bits)
Cuello de botella	El bus compartido	El directorio centralizado
Arquitectura	UMA/SMP, compartido	bus CC-NUMA, red escalable, multicore L3
Ejemplo	Intel Core i7 (entre nucleos)	Intel Core i7 (L3), AMD Opteron

Clasificación Directorio

Directorio Centralizado

Con una entrada para cada uno de los bloques (líneas) de memoria, con información de estado e información de las cachés con copia. Este directorio centralizado atiende todas las peticiones → cuello de botella.

Distribuido entre módulos de memoria principal:

Cada módulo tiene asociado un subdirectorio con la información relativa a los bloques de memoria de dicho módulo. Los subdirectorios atenderán en paralelo las peticiones de acceso.

- Directorio de vector de bits completo
- Directorio de vector de bits asignados a grupos
- Directorio limitado

Distribuido entre módulos de memoria y cachés:

Además de distribuir las filas del directorio en memoria principal, se distribuye cada una de las filas entre las cachés con copia del bloque.

- Directorio encadenado

4.12 Metodología de ejercicios de traza

Nota de examen Metodología general para trazas de coherencia

Para resolver cualquier ejercicio de traza de coherencia en el examen:

1. Leer el estado inicial de todas las caches (estado por línea) y la memoria.
2. Para cada evento en orden temporal:
 - (a) Identificar el procesador y la operación (lectura/escritura).
 - (b) Identificar el bloque afectado (a partir de la dirección y el tamaño de línea).
 - (c) Consultar el estado actual de ese bloque en ese procesador.
 - (d) Aplicar la regla de transición correcta de la tabla MSI o MESI.
 - (e) Actualizar el estado en ese procesador Y en todos los demás que espían el bus (incluyendo la memoria).
 - (f) Clasificar el acceso: acierto o fallo; tipo de fallo.
3. Rellenar la tabla resumen con los estados antes y después de cada evento.

4.13 Ejercicios resueltos con metodologia de examen

Ejercicio resuelto 1 — Traza MSI con falsa y verdadera comparticion

Enunciado. Dos procesadores P1 y P2. Tamano de linea: 2 palabras. Las variables w_1 y w_2 estan en el **mismo bloque** de memoria. Ambas caches parten vacias (todas las lineas en estado I). Protocolo MSI.

Cuadro 4.10: Secuencia de eventos

Tiempo	P1	P2
1	Escribe w_1	—
2	—	Lee w_2
3	Escribe w_1	—
4	—	Escribe w_2
5	Lee w_2	—

Solucion detallada:

t=1: P1 escribe w_1 .

Estado P1: I. Fallo de escritura. P1 genera PtLecEx. Ambas caches vacias; la memoria sirve el bloque. P1 escribe y pasa a **M**. Memoria queda invalida.

Tipo: **fallo obligatorio** (primera vez que se accede a este bloque).

t=2: P2 lee w_2 .

Estado P2: I. Fallo de lectura. P2 genera PtLec. P1 espía el bus: tiene el bloque en M, por lo que genera **RpBloque** (deposita el bloque en el bus con el valor actualizado de w_1). P1 pasa a **S**. La memoria recibe el bloque y pasa a valida. P2 recibe el bloque y pasa a **S**.

Tipo: **fallo obligatorio** en P2.

t=3: P1 escribe w_1 .

Estado P1: S. Acierto de escritura, pero P1 no tiene acceso exclusivo. Genera PtLecEx. P2 espía el bus: tiene el bloque en S, lo invalida (pasa a **I**). Memoria pasa a invalida. P1 pasa a **M**.

Tipo: **acierto con invalidacion**.

t=4: P2 escribe w_2 .

Estado P2: I. Fallo de escritura. P2 genera PtLecEx. P1 tiene el bloque en M: genera RpBloque, pasa a **I**. P2 recibe el bloque (con el w_1 actualizado de P1), escribe w_2 y pasa a **M**. Memoria invalida.

Tipo: **fallo de falsa comparticion**. P2 necesitaba w_2 , que no habia cambiado desde el punto de vista logico de P2. Sin embargo, la linea fue invalidada porque P1 escribio w_1 (una variable diferente en el mismo bloque). P2 se penaliza por la modificacion de una variable que no le compete.

t=5: P1 lee w_2 .

Estado P1: I. Fallo de lectura. P1 genera PtLec. P2 tiene el bloque en M: genera RpBloque, pasa a **S**. Memoria valida. P1 pasa a **S**.

Tipo: **fallo de comparticion verdadera**. P1 lee w_2 que fue escrita por P2 en $t=4$. Es una comunicacion real de datos: P1 genuinamente necesita el nuevo valor.

Tabla resumen:

Cuadro 4.11: Traza completa de estados MSI — Ejercicio 1

t	Evento	C(P1)	C(P2)	Mem	Senal bus	Tipo
0	inicio	I	I	V	—	—
1	P1 escribe w_1	M	I	Inv	PtLecEx	Obligatorio
2	P2 lee w_2	S	S	V	PtLec/RpBloque	Obligatorio (P2)
3	P1 escribe w_1	M	I	Inv	PtLecEx	Acierto+Inval
4	P2 escribe w_2	I	M	Inv	PtLecEx/RpBloque	Falsa comp.
5	P1 lee w_2	S	S	V	PtLec/RpBloque	Verdadera comp.

Ejercicio resuelto 2 — Traza MSI con caches no vacias y ciclos de parada

Enunciado. SMP con protocolo MSI, politicas write-allocate y write-back. Latencias: N_{memory} (fallo satisfecho por memoria), N_{cache} (fallo satisfecho por cache de otro procesador), $N_{\text{invalidate}}$ (acierto de escritura con invalidacion), $N_{\text{writeback}}$ (post-escritura por conflicto).

Estado inicial de caches:

Cuadro 4.12: Estado inicial de P0 y P1

P0	Lineas de P0				Lineas de P1			
	B0	B1	B2	B3	B0	B1	B2	B3
Etiqu.	0x100	0x108	0x110	0x120	0x122	0x108	0x110	0x120
Estado	I	S	M	I	S	S	I	M

Apartado a): P0 lee 0x100, luego P0 escribe 0x100.

Evento 1: P0 lee 0x100. B0 de P0 tiene etiqueta 0x100 en estado I: fallo de lectura. P0 genera PtLec. P1 no tiene etiqueta 0x100 en ninguna linea valida. La memoria sirve el bloque. B0-P0 pasa a S. Ciclos: N_{memory} .

Evento 2: P0 escribe 0x100. B0 de P0 en S: acierto de escritura pero genera PtLecEx. P1 no tiene 0x100: la invalidacion no afecta a ninguna cache (pero igualmente se emite). B0-P0 pasa a M. Ciclos: $N_{\text{invalidate}}$.

Total a): $N_{\text{memory}} + N_{\text{invalidate}}$

Apartado b): P0 lee 0x120, luego P0 escribe 0x120.

Evento 1: P0 lee 0x120. B3 de P0 tiene etiqueta 0x120 en estado I: fallo. P1 tiene B3 con etiqueta 0x120 en estado M. P1 genera RpBloque: deposita el bloque en el bus y pasa a S. Memoria valida. B3-P0 pasa a S. Ciclos: N_{cache} (satisfecho por cache de P1).

Evento 2: P0 escribe 0x120. B3 de P0 en S: genera PtLecEx. P1 tiene B3 en S: lo invalida (pasa a I). B3-P0 pasa a M. Ciclos: $N_{invalidate}$.

Total b): $N_{cache} + N_{invalidate}$

Apartado c): P0 lee 0x100, luego P0 lee 0x120.

Evento 1: P0 lee 0x100. Fallo en B0-P0 (I). Memoria sirve. Ciclos: N_{memory} .

Evento 2: P0 lee 0x120. Fallo en B3-P0 (I). P1 tiene B3=0x120 en M (aun, no ha cambiado). P1 proporciona el bloque. Ciclos: N_{cache} .

Total c): $N_{memory} + N_{cache}$

Ejercicio resuelto 3 — Traza MESI con caches no vacías

Enunciado. Mismas caches del Ejercicio 2, protocolo MESI. Las direcciones 0x100712 y 0x100713 pertenecen al bloque de etiqueta 0x100710.

Evento 1: P0 lee 0x100713.

El bloque 0x100710 está en B2-P0 con estado **M**. Acierto de lectura en M. Sin tráfico de bus. B2-P0 permanece en M.

Evento 2: P0 escribe 0x100712.

El bloque está en B2-P0 en estado **M**. Acierto de escritura en M. **Sin tráfico de bus:** P0 ya tiene acceso exclusivo. B2-P0 permanece en M con el nuevo valor.

Evento 3: P1 lee 0x100712.

B2-P1 tiene etiqueta 0x100710 en estado **I**: fallo de lectura. P1 genera PtLec. P0 espía el bus: tiene el bloque en M, genera RpBloque. P0 pasa a **S**. Memoria recibe el bloque y queda válida. P1 pasa a **S**.

Estado final: P0-B2 = S; P1-B2 = S; Memoria(0x110) = válida.

Cuadro 4.13: Traza del Ejercicio 3 (protocolo MESI)

Paso	Acción	P0-B2	P1-B2	Mem	Señal bus
0	inicio	M	I	Inv	—
1	P0 lee 0x100713	M	I	Inv	Ninguna
2	P0 escribe 0x100712	M	I	Inv	Ninguna (M->M)
3	P1 lee 0x100712	S	S	V	PtLec / RpBloque

Ventaja del estado M: los eventos 1 y 2 no generan tráfico de bus porque P0 ya tiene acceso exclusivo. Con MSI, si el bloque hubiera estado en S, el evento 2 habría requerido PtLecEx. Con MESI, si hubiera estado en E, la transición E→M del evento 2 también sería silenciosa.

4.14 Resumen y conclusiones del Tema 4

1. **El problema de coherencia** surge en multiprocesadores cuando varias caches privadas tienen copias del mismo dato y una lo modifica. Afecta a sistemas con escritura directa y con post-escritura.

2. **Tres condiciones de coherencia:** preservar el orden del programa, vista coherente de la memoria, y serializacion de escrituras.
3. **WI es mas eficiente que WU** para multiples escrituras al mismo bloque y es la base de los protocolos de snooping modernos.
4. **MSI:** tres estados (M, S, I). Basado en post-escritura mas invalidacion. Toda escritura en estado S genera PtLecEx aunque no haya otras copias. Ineficiente para datos privados.
5. **MESI:** anade el estado E (Exclusivo). La transicion E→M es silenciosa (sin trafico de bus). Optimo para datos usados por un solo procesador.
6. **Falsa comparticion:** fallo evitable. Dos variables sin relacion en el mismo bloque se invalidan mutuamente. La solucion es el padding o la alineacion correcta de los datos.
7. **Comparticion verdadera:** fallo inevitable. Representa comunicacion real de datos entre procesadores.
8. **Protocolos de directorio:** eliminan la difusion del snooping enviando invalidaciones solo a los nodos con copia. Escalan mejor pero tienen mayor latencia y overhead de memoria.
9. **Intel Core i7:** usa coherencia por directorio centralizado para la cache L3 compartida entre nucleos (MESIF).

Reglas rapidas para el examen — Tema 4

Estados MSI: M = unica copia valida, memoria obsoleta. S = multiples copias, todas validas. I = no presente o invalidado.

Estado adicional MESI: E = unica copia, memoria SI valida. Escritura en E pasa a M **sin senal en bus**.

Transiciones clave:

Estado	Accion	Senal en bus	Nuevo estado
I	PrLec	PtLec	S (o E en MESI si nadie mas tiene copia)
I	PrEsc	PtLecEx	M
S	PrLec	Ninguna	S
S	PrEsc	PtLecEx	M
M	PrLec/PrEsc	Ninguna	M
M	PtLec bus	RpBloque	S
M	PtLecEx bus	RpBloque+invalida	I
E	PrEsc	Ninguna	M
E	PtLec bus	Ninguna (o informa)	S
E	PtLecEx bus	Invalida	I

Tipos de fallo:

- **Obligatorio:** primera vez que se accede al bloque.

- **Falsa comparticion:** invalidado por escritura en variable diferente del mismo bloque.
- **Comparticion verdadera:** lectura de dato escrito por otro procesador (comunicacion real).